

1. Nível de dificuldade escolhido : "I am Death Incarnate!"

2. Partes não realizadas ou com limitações

- **A1** (`cliente servidor sem front-end`): Realizado sem limitações.
 - **A2** (`cliente-> front-end -> servidor`): Realizado sem limitações.
 - **B1** (`read e put com servidores replicados`): Realizado sem limitações.
 - **B2** (`take com servidores replicados`): Realizado sem limitações.
 - **C1** (`take com expressões regulares`): Realizado sem limitações.
 - **C2** (`otimizações de desempenho do algoritmo Xu-Liskov`): Realizado sem limitações.
-

3. Fases do Projeto

B2 - take concorrente

i) O *take* suporta operações concorrentes através da utilização do **algoritmo de Maekawa**, no qual cada cliente contacta um subconjunto de servidores (*voter set*) para obter permissão de acesso ao tuplo pretendido. Os acessos concorrentes aos tuplos são geridos com um sistema de locking por tuplo, implementado através de objetos *ConcurrentKeyStore* guardados num *ConcurrentHashMap*. Este mecanismo garante que apenas um cliente pode aceder a um determinado tuplo de cada vez, permitindo simultaneamente que múltiplos takes concorrentes ocorram de forma segura, desde que incidam sobre tuplos distintos. A coordenação e recolha dos grants está centralizada na classe *FrontEndServiceImpl*. Caso o tuplo pretendido ainda não exista, o cliente aguarda até que este seja introduzido e libertado com um *notifyAll*.

C1 - take com expressões regulares

i) A introdução de expressões regulares altera significativamente o funcionamento do *take*, uma vez que deixamos de procurar por um tuplo exato e passamos a procurar por qualquer tuplo que corresponda à expressão. Assim, o Frontend necessita de garantir que todos os tuplos matching nas várias réplicas estão efetivamente bloqueados antes de poder realizar a operação de remoção.

Este novo comportamento introduz desafios adicionais de concorrência: os tuplos que satisfazem a expressão podem estar bloqueados devido a pedidos de outros clientes. Assim, as réplicas do *voter set* só enviam o *grant* quando consegue bloquear todos os tuplos relevantes. Só após isso o Frontend realiza a interseção entre os tuplos bloqueados. Caso a interseção seja nula, os tuplos já reservados são libertados, e o Frontend aguarda (via *wait()*), voltando a tentar mais tarde, com um mecanismo de backoff exponencial. Se a interseção for válida, o *take* é realizado com o primeiro tuplo comum.

O algoritmo segue os seguintes passos:

1. O Frontend envia um *grantRequest* com a expressão regular.
2. Cada réplica tenta bloquear todos os tuplos locais que fazem match com a *regex*, utilizando *ServerState.requestAccess()*.
3. Se uma réplica não conseguir bloquear todos os tuplos relevantes, liberta os que tinha reservado e entra em espera (*wait()*).
4. Quando cada réplica consegue bloquear os tuplos correspondentes, envia o respetivo *grant* para o *FrontEndServiceImpl*.
5. O *FrontEndServiceImpl* verifica se existe interseção entre os tuplos bloqueados por cada réplica. Se a interseção for nula, os tuplos são libertados (via *ServerState.release()*), e o

cliente tenta novamente após um tempo de espera.

6. Se existir um tuplo comum à interseção, esse é removido, e os restantes tuplos são desbloqueados.

A coordenação correta das respostas das réplicas é garantida por um mapeamento consistente entre respostas e réplicas.

C2 - Otimizações de desempenho (Xu-Liskov)

i) O objetivo das otimizações implementadas é reduzir o tempo de espera dos clientes e aumentar a eficiência do sistema. Para isso, algumas operações passaram a ser realizadas em segundo plano. No caso de um pedido de put, o Frontend responde automaticamente ao cliente (*onCompleted* e *onNext*) dizendo que o comando foi executado, mesmo que o pedido só seja enviado às réplicas depois disso, otimizando o tempo de resposta imediato. Para os pedidos de take, o frontend responde ao cliente (*onCompleted* e *onNext*) assim que identifica o tuplo que será retirado, ou seja, assim que é garantida uma interseção válida dos tuplos bloqueados nas réplicas do voter set. Isto permite que a resposta seja enviada rapidamente, sem esperar pela execução completa do *take* nas réplicas.

ii) Para mitigar os problemas das otimizações, garantimos que, em cada réplica, os pedidos dos clientes são executados na ordem correta. Isto é feito através de um sistema de senhas por cliente (*clientTickets*), em que o Frontend atribui uma senha a cada pedido e as réplicas controlam a execução com base na ordem dessas senhas.

Além disso, garantimos que o pedido *put* só é executado após todos os pedidos *take* do respetivo cliente serem concluídos: implementámos um contador (*clientTakeInProgress*) no Frontend, que é incrementado para cada *take* recebido e decrementado quando esse take termina em todas as réplicas. Quando um *put* é recebido, o sistema verifica o número de *takes* em execução e entra num loop de espera (*lock wait*) até que todos os *takes* anteriores sejam concluídos (*notifyAll*), garantido assim que a restrição é respeitada.

4. Testes incluídos

Os testes incluídos na pasta `testes-finais` cobrem:

- Teste1: Teste de Take Regex com Interseção Nula durante N tentativas
- Teste2: Teste de Linearidade
- Teste3: Teste de Linearidade
- Teste4: Teste de Take Regex com Interseção Nula durante N tentativas
- Teste5: Teste de Take Regex para cálculo da Interseção
- Teste6: Teste de Linearidade
- Teste7: Teste de Linearidade
- Teste8: Teste de Linearidade

5. Outras notas e comentários

- Implementámos uma estrutura de `ConcurrentKeyStore` que associa tuplos a contadores e locks, simplificando a gestão de concorrência.
- O sistema foi testado com delays simulados e “race conditions” entre clientes.
- Tivemos em conta várias situações-limite, como interseções vazias entre servidores ou tentativas de desbloqueios não autorizados por threads erradas.
- O comportamento expectável de cada teste está especificado no respetivo ficheiro de output.